



3. Einfache Datentypen und sequentielle Anweisungen

Blöcke

Variablendeklarationen

Ausdrücke

Zuweisungen



3.1 Blöcke



Aufbau eines Java-Programms

Zur Zeit haben Ihre Java-Programme immer folgenden Rahmen:

```
1 package de.ostfalia.gdp.ws16.si; //max. ein Paket
2 import java.util.Scanner;        //möglicherweise mehrere
3                                 //zu importierende Klassen
4
5 public class MyClass {
6
7     //Hier steht möglicherweise später weiterer Code dieser Klasse
8
9     public static void main(String[] args) {
10
11         //Hier steht der Code der main-methode,
12         //mit der die Ausführung beginnt
13
14     }
15
16     //Hier steht möglicherweise später weiterer Code dieser Klasse
17
18 }
```



Blöcke

Anweisungen können hintereinander geschrieben werden.

Mehrere Anweisungen können durch Klammern (`{`, `}`) zu einem Block zusammengefasst werden.

Beispiel:

```
1  {  
2      result2 = (cost * days * dist2) / (dist1 + dist2);  
3      result1 = cost * days - result2;  
4      System.out.println("Kosten_Fahrer:_" + result1);  
5      System.out.println("Kosten_Mitfahrer:_" + result2);  
6  }
```

Syntax:

$$\begin{aligned} \textit{Statement} &:= ; \mid \textit{Block} \mid \dots \\ \textit{Block} &:= \{ \textit{BlockStatement}^* \} \\ \textit{BlockStatement} &:= \textit{LocalVarDeclStmt} \mid \textit{Statement} \end{aligned}$$



Syntaktische Elemente von Programmen

Wir kennen schon

- **Datentypen** (types): `int`, `double` *Type*
- **Bezeichner** (Identifier) selbstdefinierte Zeichenfolgen *Identifier*
- **Variablendeklarationen** (Variable declaration) `int x;`
VariableDeklaration
- **Ausdrücke** (Expressions) `3+x` *Expression*
- **boole'sche Ausdrücke** (boolean Expressions) `(3>1)` *BoolExpression*
- **Anweisung** (Statement) *Statement*
- **Anweisungsfolge** (Statement list) *StatementList*



3.2 Variablendeklarationen



Lokale Variablendeklaration I

Erinnerung: In Java gehören Variable zu einem bestimmten **Datentyp**.

Jeder **Datentyp** setzt sich aus folgenden Bestandteilen zusammen:

1. Die Menge der Werte, d.h. der **Wertebereich**.
2. Die **Darstellungen** für die Werte.
Man unterscheidet zwischen der Darstellung im Programmtext (Literele) und der im Speicher.
3. Die **Methoden** bzw. **Operatoren**, die für diesen Typ definiert sind.

Beispiel:

```
int a, b, c=1;
```



Lokale Variablendeklaration II

Syntax:

```
LocalVarDeclStmt  := Type VariableDeclarators ;  
                   Type  := byte | short | char | int | long | float  
                           | double | boolean  
VariableDeclarators := VariableDeclarator  
                           | VariableDeclarator , VariableDeclarators  
VariableDeclarator := Identifier | Identifier = Expression
```

- Im Moment verwenden wir nur die **primitiven Datentypen** byte, short, char, int, long , float, double, boolean
- Konvention: Lokale Variablen beginnen mit Kleinbuchstaben.
- Konvention: der Name einer Variablen “erklärt” ihre Bedeutung: summe, zeit,..., nicht x15a, y1.



Lokale Variablendeklaration III

Datentyp	Bemerkung	Min	Max
byte	8 Bit, 2er-Komplement	-128	+127
short	16 Bit, 2er-Komplement	-32.768	+32.767
int	32 Bit, 2er-Komplement	-2.147.483.648	+2.147.483.647
long	64 Bit, 2er-Komplement	-9.223.372.036.854.775.808	+9.223.372.036.854.775.807
float	32-bit IEEE 754 floating point		
double	64-bit IEEE 754 floating point		
boolean	Größe nicht spezifiziert	false	true
char	Ein 16-bit Unicode Zeichen	'\u0000' (oder 0)	'\uffff' (oder 65.535)

Abbildung 8: nach S. Fischer, Java Einführung



Bezeichner: Java-Identifizier

Bestehen aus

- Unicode Buchstaben
- digitalen Ziffern
- Währungssymbolen
- Verbindungszeichen (Bsp.: Unterstrich “_”)
- dürfen theoretisch beliebig lang sein

Einschränkungen

- Beginnen mit einem Buchstaben, Währungssymbol oder Verbindungszeichen, d.h. insb. **nicht mit einer Ziffer**
- enthalten **keine Leerzeichen**
- Identifizier sind case-sensitive (Unterscheidung Groß-/ Kleinschreibung)
- und sind **keine reservierten Schlüsselwörter** (class, main, for,...)
Details unter http://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html



Legale Bezeichner

- Test
- __DAS_IST_S000_COOL__
- bóolêáñ
- DAGOBERT\$\$\$
- \$€

Illegale Bezeichner

- 2cool4you
- nein danke
- yolo!
- private



Konventionen

Zum besseren Lesen von Programmen halten sich Entwickler an Code-Konventionen:

- hier: an den sog. Java Code Style
- Java Code Style für Identifier (Anfang)
 - Bezeichner für Variablen beginnen mit einem Kleinbuchstaben
 - "Camel Case" Schreibung: `eineVariableFuerEineReelleZahl`
- Für internationale Zusammenarbeit: auf ASCII beschränken (keine Umlaute)
- Details unter <http://www.oracle.com/technetwork/java/javase/documentation/codeconventions-135099.html#367>



Variablendeklaration

```
1 public class LocVarDeclaration {  
2  
3     public static void main(String[] args) {  
4         // TODO Auto-generated method stub  
5         byte einNettesByte;  
6         short alberich;  
7         int trois = 3;  
8         long dirkNowitzky;  
9         float oker;  
10        double distance2Moon = 280000.0;  
11        boolean wahr;  
12        char alpha = 'a';  
13    }  
14  
15 }
```

Was passiert, wenn Sie dieses Programm laufen lassen?

Was ist die Semantik?



Semantik: Lokale Variablendeklaration, Gültigkeitsbereich

- Für **lokale Variablen** wird zur Laufzeit (=bei der Ausführung) Speicherplatz auf dem Stack vorgesehen, sobald die Ausführung des Blocks beginnt, in dem sie deklariert sind:
sie werden gestapelt.
- Sobald die Ausführung des umgebenden Block endet, wird der Speicher für lokale Variablen wieder freigegeben.
- Der Zeitraum zwischen Betreten und Verlassen des Blocks heißt die **Lebensdauer** der Variablen.
- Freigegebener Speicher kann für andere Variablen genutzt werden.
- Lokale Variablen müssen vom Benutzer initialisiert werden.

Ein Block ist der **Gültigkeitsbereich (scope)** der in ihm deklarierten lokalen Variablen.

Was passiert, wenn Blöcke geschachtelt werden?



Gültigkeitsbereiche

```
1 public class NestedBlocks {  
2     public static void main(String[] args) {  
3         int x = 1;  
4         {  
5             int x; // neue Variable, ebenfalls x  
6             int y = 2;  
7             x = 3;  
8             System.out.println(x);  
9             System.out.println(y);  
10        }  
11        System.out.println(x);  
12        {  
13            int y = 4;  
14            System.out.println(y);  
15        }  
16        System.out.println(y);  
17    }  
18 }
```



Beispiel Gültigkeitsbereiche

```
1 package de.ostfalia.gdp.ws15.si;
2
3 public class NestedBlocks {
4     public static void main(String[] args) {
5         int x = 1;
6         {
7             Duplicate local variable x // neue Variable , heißt auch x
8             int y = 2;
9             x = 3;
10            System.out.print(x);
11            System.out.print(y);
12        }
13        System.out.print(x);
14        {
15            int y = 4;
16            System.out.print(y);
17        }
18        System.out.print(y);
19    }
20 }
21
```

Abbildung 9:



Gültigkeitsbereiche II

- Java verbietet die Deklaration einer lokalen Variablen mit identischem Namen innerhalb eines Blocks.
⇒ Zeile 5: **Fehler: Duplicate local variable x**
- Zeile 10 schließt den inneren Block (Zeile 4-10), der Speicher für y wird freigegeben, und in Zeile 13 wird Speicher für eine **neue** Variable y angelegt und mit 4 belegt.
- Zeile 15 schließt den inneren Block (Zeile 12-15), der Speicher für y wird freigegeben. und in Zeile 17 ist kein y sichtbar.
⇒ Zeile 17: **Fehler: y cannot be resolved to a variable**
- Konvention: Für Variablen, die nur kurzzeitig (ein paar Code-Zeilen lang) verwendet werden, wird ein umgebender Block angelegt, der anzeigt, dass ihre Lebensdauer begrenzt ist.

Achtung: Diese Gültigkeitsregel gilt nur für lokale Variablen!



Konstantendeklaration

Werte, die mehrfach vorkommen, und sich während des Programmlaufs nicht ändern, heißen **Konstanten**.

- Konstanten werden wie Variablen deklariert. Schlüsselwort `final`
- Konvention: Verwenden Sie GROSSBUCHSTABEN oder GROSS_BUCHSTABEN als Konstantennamen.

Beispiel:

```
1  final double PI = 3.14;  // pi
2
3  final double MWST = 19.0; //Mehrsteuer
```



Konstantenverwendung

```
1 public class GasPreis {
2     public static void main(String[] args) {
3         final String KVT      = "Kleinverbrauchertarif";
4         final String NT       = "Normaltarif";
5         //...
6         final int PCT         = 100;
7
8         final int KVGRUNDPREISWF = 4398;
9         final int KVARBEITSPREISWF = 713;
10        final int NTGRUNDPREISWF = 8002;
11        final int NTARBEITSPREISWF = 610;
12        //...
13
14        int kw = in.nextInt();
15
16        if (kw >= 0) {
17
18            int kvWF = KVGRUNDPREISWF + (KVARBEITSPREISWF * kw) / PCT;
19            int ntWF = NTGRUNDPREISWF + (NTARBEITSPREISWF * kw) / PCT;
20            //...
```



Konstantenverwendung

- **Separation of Concerns:** Trennung von
 - Festlegung numerischer Werte
 - Programmablauf und Berechnungen
- **Vorteil: Mehr Übersicht!**
 - **beim Schreiben**
 - **bei der Fehlersuche**
 - **bei Änderungen**



3.3 Ausdrücke



Ausdrücke I

Beispiel: Ausdrücke

```
1  public static void main(String[] args) {  
2  
3      int a, b;  
4      int u = 9;  
5      int v = 3;  
6      int w = 8;  
7  
8      a = u * v / w + v;  
9      System.out.println(a);  
10 }
```



Ausdrücke II

- Ein **Ausdruck** besteht aus einem **Operator** und **Operanden**. Die Anzahl der Operanden heißt **Stelligkeit** des Operators. Die meisten Operatoren in Java sind ein- oder zweistellig. Der Fragezeichen-Operator ist dreistellig.
- Die Operanden müssen von einem bestimmten, vorgegebenen Typ sein. In manchen Fällen werden implizit oder vom Entwickler explizit Typumwandlungen vorgenommen werden.
- Jeder Ausdruck hat einen bestimmten **Rückgabewert**, der durch Operator und Operanden festgelegt wird.
- **Prioritäts-** und **Assoziativitätsregeln** regeln die Auswertungsreihenfolge. Durch Klammerung kann jede beliebige Reihenfolge erreicht werden.
- Ausdrücke können **Seiteneffekte** besitzen, d.h. den **Zustand** verändern.



Ausdrücke III

- Die Reihenfolge der Auswertung der Operanden ist festgelegt: Der **linke Operand eines Ausdrucks ist vollständig vor dem rechten Operanden auszuwerten.**



Arithmetische Operatoren

Operator	Stelligkeit	Bedeutung	Bemerkung
+	1	pos. Vorzeichen	auf ganzen Zahlen ganzzahlig! ⁸
-	1	neg. Vorzeichen	
+	2	Summe	
-	2	Differenz	
*	2	Produkt	
/	2	Quotient	
%	2	Rest, mod	
++	1	Präinkrement	
++	1	Postinkrement	
--	1	Prädekrement	
--	1	Postdekrement	

Details finden Sie unter

<http://openbook.rheinwerk-verlag.de/javainsel/>, im Kap. 2.4.2

⁸auch für Gleitkommazahlen, siehe fmod in C



Post-/Prä- In/Dekrement

Das Erhöhen bzw. Verringern einer Variable um 1 heißt **Inkrementieren** und **Dekrementieren**. Dafür gibt es die unären Operatoren ++ bzw. --, die vor oder hinter einer Variablen stehen können:

Operator	Langform	Name
b=++a;	a=a+1; b=a;	Präinkrement
b=--a;	a=a-1; b=a;	Prädecrement
b=a++;	b=a; a=a+1;	Postinkrement
b=a--;	b=a; a=a-1;	Postdecrement



Vergleichsoperatoren

Operator	Stelligkeit	Bedeutung	Bemerkung
<code>==</code>	2	Gleich	Für Referenztypen <code>true</code> , falls beide Werte auf dasselbe Objekt zeigen.
<code>!=</code>	2	Ungleich	Für Referenztypen <code>true</code> , falls beide Werte auf verschiedene Objekte zeigen.
<code><</code>	2	Kleiner	
<code><=</code>	2	Kleiner gleich	
<code>></code>	2	Größer	
<code>>=</code>	2	Größer gleich	

Vergleichsoperatoren ergeben einen Wert vom Typ `boolean`.



Logische Operatoren

Operator	Stelligkeit	Bedeutung
!	1	logisches NICHT (Negation)
&&	2	UND mit Short-Circuit-Evaluation
	2	ODER mit Short-Circuit-Evaluation
&	2	UND ohne Short-Circuit-Evaluation
	2	ODER ohne Short-Circuit-Evaluation
^	2	exklusives ODER

Bei einer **Short-Circuit-Evaluation** wird der zweite Operand nur ausgewertet, wenn er das Ergebnis noch ändern kann.

Negation ! hat höhere Priorität als UND und ODER.



Wertetabelle

Den Wert eines booleschen Ausdrucks kann man mit einer Wertetabelle darstellen:

x	y	$x \& \& y$	$x \parallel y$	$x \hat{\ } y$	$!x$	$!x \& y$
false	false					
false	true					
true	false					
true	true					



Beispiel: Boolesche Ausdrücke

Ist der Ergebnistyp eines Ausdrucks `boolean`, sprechen wir von einem **booleschen Ausdruck**.

Die Vergleichsoperatoren sind auch auf `char` anwendbar:

```
1  char c= ...;
2
3  boolean isCapitalLetter = (c >= 'A') && (c <= 'Z');
4  boolean isWhitespace = (c == ' ') || (c == '\n') || (c == '\t');
```



Ergebnistyp

- Jedes Ergebnis eines Teilausdrucks hat einen **Ergebnistyp**.
- **Auswertungsreihenfolge**:
 - von links nach rechts
 - unter Beachtung des Operatorenrangs, d.h. insb. auch Punkt-vor-Strich-Rechnung.
- Wenn ein (Teil-)Ausdruck nur ganze Zahlen⁹ verwendet, ist der Ergebnistyp `int`, es sei denn ein Operand ist vom Typ `long`, dann ist auch der Ergebnistyp `long`.
- Bei gemischt getypten Ausdrücken, ist das Ergebnis vom allgemeineren Typ, wobei gilt

$$\text{byte} \subseteq \text{short} \subseteq \text{int} \subseteq \text{long} \subseteq \text{float} \subseteq \text{double}$$

- Konstanten sind im ganzzahligen Fall vom Typ `int`, Fließkommazahlen vom Typ `double`.

Konsequenz : **Kommutativgesetz** gilt manchmal unerwartet nicht für Java-Ausdrücke: $5/6 * 1.0 \neq 1.0 * 5/6$

⁹`byte`, `short`, `int` in beliebiger Kombination



Zuweisungsoperatoren

Operator	Stelligkeit	Bedeutung	Bemerkung
=	2	Einfache Zuweisung	a = b liefert b, weist b an a zu
+=	2	Additionszuweisung	a += b liefert a+b, weist a+b an a zu
-=	2	Subtraktionszuweisung	analog
*=	2	Multiplikationszuweisung	analog
/=	2	Divisionszuweisung	analog
&=	2	UND-Zuweisung	analog
...	2	...	analog

Eine **Zuweisung** ist ein Ausdruck mit einem **Seiteneffekt**. Sie liefert einen Rückgabewert.



Beispiel: Zuweisungen und Links-nach-Rechts-Auswertung

Falls die Variable `i` den Wert 2 besitzt, dann ergibt die Auswertung des Ausdrucks

$$(i=3) * i$$

den Wert 9 und nicht 6.

Falls die Variable `j` den Wert 2 besitzt, dann ergibt die Auswertung des Ausdrucks

$$j * (j=3)$$

den Wert 6. Diese Ausdrücke besitzen einen **Seiteneffekt**.



Operatoren für Zeichenketten

Das **+** wird auch zur **Konkatenation von Zeichenketten** verwendet:

Zeichenketten

```
1 System.out.println("Das_Ergebnis_ist_" + result);  
2 System.out.println("e=" + e + "_", "f=" + f);
```

Aber Vorsicht::

```
1     int a = 47;  
2     int b = 35;  
3  
4     System.out.println(a+b+"x");  
5     System.out.println("x"+a+b);  
6  
7  
8 >>> 82x  
9 >>> x4735
```



Weitere Operatoren I

- Bitweise Operatoren: `|` `&` `^` `>>` `>>>` `<<`

Mithilfe der bitweisen Operatoren kann auf die Binärdarstellung von primitiven Datentypen zugegriffen werden. Ein Wert wird dabei als Folge von Bits angesehen, die mit den Operatoren bitweise manipuliert werden können.

`w>>a`: `w` als Bitfolge interpretiert und Verschiebung um `a` Stellen nach rechts. (Vorzeichen wird beibehalten)

`w>>>a`: `w` als Bitfolge interpretiert und Verschiebung um `a` Stellen nach rechts. (Auffüllen mit Nullen).

`w<<a`: `w` als Bitfolge interpretiert und Verschiebung um `a` Stellen nach links.



Weitere Operatoren II

Bitweise Operatoren: | & ^ >> >>> <<

```
1  byte n = 1;
2  int m = (n << 7);
3  byte k = (byte) (n << 7);
4  m = m << 24;
5  int l = (m >> 1);
6  l = (m >>> 1);
7
8
9  byte b0 = 0;
10 byte b = 0b00001111;
11 b0 = (byte) (23 & b);
```

Anmerkung: b nennt man eine „**Bitmaske**“. Anwendung: systemnahe Programmierung oder effiziente Behandlung kleiner Mengen.



Weitere Operatoren III

- **Fragezeichen-Operator:** `?` `:`

Der Fragezeichen-Operator ist der einzige dreistellige Operator in Java. Er erwartet einen logischen Ausdruck und zwei weitere Ausdrücke, die entweder beide numerisch, vom Typ `boolean` oder aber beide von einem Referenztyp sein müssen.

`a ? b : c` liefert `b`, falls `a` wahr ist, und `c`, falls `a` falsch ist.
Der Ergebnistyp ist der allgemeinere der beiden Typen von `b` und `c`.

Beispiel:

```
minVal = (a < b) ? a : b;
```



Weitere Operatoren IV

- Type-Cast-Operator

Der Ausdruck (type) a wandelt den Ausdruck a in einen Ausdruck vom Typ type um. Das Ergebnis ist ein Ausdruck, der nur noch rechts von einem Zuweisungsoperator stehen darf.

Der Type-Cast-Operator wird angewendet, wenn der Compiler keine oder nicht die gewünschten impliziten Typumwandlungen vornimmt.

Beispiel Typecast

```
1  byte i;  
2  long y = (int) 4.0;  
3  int z = (int) y;  
4  float f = (float) 1.0;  
5  short d = (byte) ((i=3) * i);
```



Rang der Operatoren I

Operator	Rang	Typ	Beschreibung
++, --	1	arithmetisch	Inkrement und Dekrement
+, -	1	arithmetisch	unäres Plus und Minus
~	1	integral	bitweises Komplement
!	1	boolean	logisches Komplement
(Typ)	1	jeder	Cast
*, /, %	2	arithmetisch	Multiplikation, Division, Rest
+, -	3	arithmetisch	Addition und Subtraktion
+	3	String	String-Konkatenation
<<	4	integral	Verschiebung links
>>	4	integral	Rechtsverschiebung mit Vorzeichenerweiterung
>>>	4	integral	Rechtsverschiebung ohne Vorzeichenerweiterung
<, <=, >, >=	5	arithmetisch	numerische Vergleiche
instanceof	5	Objekt	Typvergleich
==, !=	6	primitiv	Gleich-/Ungleichheit von Werten
==, !=	6	Objekt	Gleich-/Ungleichheit von Referenzen

Abbildung 10: aus Java ist auch eine Insel



Rang der Operatoren II

&	7	integral	bitweises Und
&	7	boolean	logisches Und
^	8	integral	bitweises Xor
^	8	boolean	logisches Xor
	9	integral	bitweises Oder
	9	boolean	logisches Oder
&&	10	boolean	logisches konditionales Und, Kurzschluss
	11	boolean	logisches konditionales Oder, Kurzschluss
?:	12	jeder	Bedingungsoperator
=	13	jeder	Zuweisung
*=, /=, %=, +=, -=, <<=, >>=, >>>=, &=, ^=, =	14	jeder	Zuweisung mit Operation

Abbildung 11: aus Java ist auch eine Insel



3.4 Zuweisungen



Zuweisungen

Syntax:

$$\begin{aligned} \textit{Statement} &:= \dots \mid \textit{Assignment} \ ; \mid \dots \\ \textit{Assignment} &:= \textit{LeftHandSide} \ \textit{AssignOperator} \ \textit{AssignExpr} \\ \textit{LeftHandSide} &:= \textit{Identifier} \mid \dots \\ \textit{AssignExpr} &:= \textit{Assignment} \mid \textit{Expression} \\ \textit{AssignOperator} &:= = \mid += \mid -= \mid *= \mid /= \mid \dots \end{aligned}$$

Beispiel:

```
1  int x,y;
2  y = 2;
3  x = 3 * y;
```



Semantik von Zuweisungen

Der Wert des Ausdrucks der rechten Seite *AssignExpr* wird ermittelt, und der Variablen der *LeftHandSide* zugewiesen.

Java verlangt, dass Ergebnistyp und Datentyp der *LeftHandSide* **zuweisungskompatibel** sind.

- In Standardfällen nimmt Java implizit einen Type-Cast vor.

```
1  short d=1;
2  int x;
3  long y;
4  x = d;
5  y = x;
```

Bei einer Zuweisung verlangt Java, dass die rechte Seite *AssignExpr* einen Wert hat, andernfalls wird ein Compilerfehler gemeldet: **variable d may not have been initialized.**

```
1  short d;
2  int x;
3  x = d;
```



Beispiel zu Zuweisungen

```
byte x;           1
x=2;              2
{                 3
    int y = 128;  4
    x = (byte) y; 5
    y += (x=10);  6
}                 7
x/=3;             8
```

Abbildung 12: Beispiel von F. Höppner



Beispiel zu Zuweisungen

```

byte x;           1
x=2;             2
{               3
    int y = 128;  4
    x = (byte) y;  5
    y += (x=10);  6
}               7
x/=3;           8
  
```

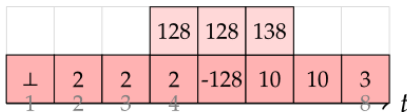


Abbildung 12: Beispiel von F. Höppner



Exkurs: Bitfelder



Exkurs: Ints und Floats

Ints2Floats and back again

```
1 int y = 1024*1024*1024 +63;
2
3 System.out.println(y);
4 //Ausgabe in Binärdarstellung
5 System.out.println(Integer.toString(y,2));
6
7 float f = y;
8 int x = y + 63;
9 int z = (int) (f + 63);
```

Wieviele Stellen dezimal/binär hat y?

Wie groß ist der Unterschied zwischen x und z? Es gilt

$$\begin{array}{ll} A: x = z & C: 10 \leq |x - z| < 100 \\ B: |x - z| < 10 & D: 100 \leq |x - z| \end{array}$$

Was folgt daraus für die Verwendung von int bzw. long und float bzw. double?